

OpenCollab SWE-bench V1 Harness 重构与评测报告

Codex

2026 年 7 月 8 日

摘要

这份报告记录 OpenCollab SWE-bench harness 重构的完整工程过程。工作从干净基础节点开始，在独立 worktree 中保留已有可用成果，随后围绕 V1 评测流程建立状态判定层、候选身份配对、有限损失恢复、单命令切片 runner、官方评测衔接、远端运行隔离、只读角色边界、工具调用边界和硬超时机制。报告同时记录 26-27、28-29、31、34-35 等实际评测切片，以及评测过程中暴露出的框架问题和对应修复。最终 34-35 切片由一条 runner 命令完成生成与官方评测，结果为 2 题生成完成、2 题官方评测完成、1 题 resolved、1 题 unresolved、0 个技术失败，并经过独立 Agent 最终复审。

1 工程背景与目标

这次工作的起点是对原有 SWE-bench 评测 harness 的一次重整。此前仓库中已经出现过可用代码，也出现过不少围绕评测 harness 的反复尝试。用户给出的核心约束很明确：先选择一个干净节点，保留真正有价值的已有内容，建立一个可以继续开发的基础节点；随后在这个基础节点上重构评测 harness。另一个并行线程仍在原工作区执行单 Agent 25 题评测，因此写入和重构都放在独立 worktree `/Users/Kai/Documents/MultiAgent/OpenCollab-harness-clean-foundation` 中完成，避免影响正在运行的评测。

重构目标集中在 V1。V2 和 V3 的实验性设计可以暂缓，V1 必须可运行、可恢复、可报告、可复审。对恢复能力的要求也从“无损恢复”调整为工程上更稳妥的有限损失恢复：长任务运行时周期性保存 worktree diff，默认 300 秒一次，因此进程异常结束时最多丢失当前模型或工具回合以及最近一个检查点间隔内的编辑。这个目标比追求完整会话状态恢复更贴近当前架构，也更容易验证。

用户还要求每次重要改动后由独立 Agent 复审。这个要求贯穿整个过程：初版状态层和 V1 入口先接受复审并修正；后续每次由真实评测暴露出的框架问题，都先修代码，再通过测试、实际运行或独立评审确认。

2 基础节点和分支隔离

干净基础选择围绕两个事实展开。首先，`cab511f` 已经包含 formal main merge 和更早的 SWE committee 工作，保留了有价值的文档、适配器与评测上下文。其次，它尚未卷入后续 harness 反复

改动造成的复杂状态。基于这个判断，在独立 worktree 中建立基础节点，并形成提交 54a6c35 建立评测重构前的可用基础。后续所有写入都发生在分支 `codex/harness-clean-foundation` 上。

这个隔离方式解决了两个实际问题。第一，原工作区可以保持只读，正在跑的并行评测不会被重构动作干扰。第二，新的分支历史能够清楚展示从基础节点到 V1 harness 的每一步变化，便于复审和回滚。

3 重构后的 V1 Harness 结构

重构后的 V1 由四类组件组成。第一类是纯状态层，负责读取预测和指标记录，判断任务是否需要生成、是否已有非空 patch、是否可以进入官方评测、是否已有评测报告。第二类是 V1 runner，负责对连续题号切片发起一次生成和一次官方评测，并输出 JSON 与 Markdown 报告。第三类是 workflow 层，负责把一个 SWE 任务拆成定位、契约、验证、编码、风险审查和最终验证等角色。第四类是运行可靠性层，包括有限损失检查点、工具执行边界、角色超时、远端代理健康检查和中断清理。

表 1: V1 harness 的主要产物

文件或模块	作用
<code>opencollab/opencollab/harness/swe_eval_records.py</code>	读取 <code>predictions.jsonl</code> 与 <code>metrics.jsonl</code> ，提取 <code>task_id</code> 、 <code>record_id</code> 、 <code>patch_sha256</code> ，用候选身份配对 prediction 与 metric，避免旧报告或旧 metric 被误用。
<code>opencollab/opencollab/harness/swe_eval_decision.py</code>	定义 <code>TaskState</code> ，把任务归为 <code>needs_generation</code> 、 <code>ready_for_eval</code> 、 <code>eval_done</code> 、 <code>technical_eval_failed</code> 等状态，并输出可机器读取的状态行。
<code>opencollab/opencollab/harness/swe_eval_discovery.py</code>	汇总 run directory 中的预测、metric 和官方评测报告，供状态层与看板使用。
<code>opencollab/opencollab/harness/swe_checkpoint.py</code>	在运行目录内保存 worktree patch 和元数据，默认 300 秒检查点，并在恢复时把最近可提交 patch 应用回环境。
<code>scripts/swe_v1_prolite_runner.py</code>	对 SWE-bench Pro-Lite 连续切片执行单命令评测：远端运行时同步、代理健康检查、逐题生成、非空 patch 过滤、官方评测、报告写入。
<code>workflows/validation_council_solve.py</code>	实现 validation council 工作流。角色包含 Localizer、Contract Miner、Test Cartographer、Validation Factory、Judge、Coder、Patch Validator、Diff Risk Auditor 和 Final Verifier。
<code>opencollab/opencollab/application/async_timeout.py</code>	提供 <code>abandon_on_timeout</code> ，当 provider coroutine 在取消清理中卡住时，调用方仍能在时间边界处返回 <code>TimeoutError</code> 。

3.1 候选身份与报告配对

原 harness 的一个高风险点是 stale report reuse: 如果只按 task id 或文件名找报告, 同一个任务的旧 patch 可能和新 patch 的报告混在一起。新的状态层要求候选身份至少包含 `record_id` 和完整 `patch_sha256`。当 prediction 有 `record_id` 时, metric 必须用相同 `record_id` 配对, 并且 patch sha 也要匹配; 当缺少 `record_id` 时, 才退回 patch sha 配对; `legacy_latest` 只留给旧记录兼容, 不作为新路径的依赖。

这个设计让“生成了什么”和“评测了什么”连在一起。报告中的每一行都可以追溯到具体 patch sha 和 record id, 因此官方评测结果不会被另一次 run 的报告污染。

3.2 有限损失恢复

有限损失恢复由 `swe_checkpoint.py` 完成。检查点保存的是当前 worktree diff, 而非模型内部状态。默认间隔为 300 秒, 元数据中记录 `loss_bound_seconds`、patch 字节数、patch sha 和是否可作为提交 patch 使用。恢复时先确认工作区干净, 再通过 `git apply --binary` 应用最近保存的 patch。

这种恢复方式有明确边界。它可以保护源码编辑结果, 不能恢复已经消耗掉的模型上下文, 也不能保证恢复到角色内部推理的中间状态。对评测 harness 来说, 这个边界是可接受的, 因为官方 eval 只需要最终 patch 和稳定记录, 而生成阶段如果异常退出, 可以从最近 patch 或显式失败状态继续处理。

3.3 单命令 V1 Runner

`scripts/swe_v1_prolite_runner.py` 是此次 V1 的核心入口。脚本接收 `--start-index` 和 `--limit`, 在远端建立隔离 run directory, 把必要代码同步到 `_runtime/repo`, 检查代理健康, 按切片逐题执行 generation, 再对每个非空 patch 执行官方评测。每个任务有一次有界生成尝试和一次有界官方评测尝试, 脚本最后写入 JSON 与 Markdown 报告。

34-35 的最终命令为:

```
python3 scripts/swe_v1_prolite_runner.py \
  --start-index 34 \
  --limit 2 \
  --run-id formal_34_35_readonlytight_20260708 \
  --max-task-starts 2 \
  --json-output docs/monitoring/swe_v1_16m_prolite34_35_readonlytight_report.json \
  --markdown-output docs/monitoring/swe_v1_16m_prolite34_35_readonlytight_report.md
```

这条命令从生成到官方评测都由脚本驱动, 中途的人工动作只用于观察是否出现新的框架问题。最终脚本自然结束并写出报告。

4 Validation Council 工作流

`validation_council_solve.py` 采用多角色顺序协作。前半段先定位问题、抽取行为契约、识别测试方式，并在编码前提出和筛选验证候选。Coder 只有在有足够上下文后写源码，并运行可用的公共测试。写完 `patch` 后，Patch Validator、Diff Risk Auditor、Post-Patch Validation 和 Final Verifier 检查 `patch` 是否只修改合法源码路径、是否留下临时文件、是否满足被接受的验证证据。

这套设计的价值在于将 SWE 任务中的“理解问题”“选择验证”“写补丁”“审查差异”分开。早期版本的失败也恰好发生在这些角色边界上：只读角色会试图寻找写工具，结构化角色在没有进展时继续读文件，硬写入阶段会声明要写但继续读，差异风险阶段会重复扫描同一证据。后续提交逐步把这些边界变成显式规则和测试。

最终版本里，非 coder 结构化角色的超时统一为 300 秒，Coder 仍保留 1800 秒。只读角色的 `prompt` 明确要求：不能创建或运行 `probe` 时，必须在 `structured_output` 中报告限制；不能去搜索不存在的写工具；临时验证文件只能由拥有命令或写入工具的角色放在 `/tmp/opencollab-validation-*` 下。Patch Validator 和 Final Verifier 可以运行既有测试，但不能创建新的 `probe` 文件，也不能寻找写工具。

5 关键提交序列

从基础节点之后，提交序列可以按工程阶段理解。早期提交建立状态层与恢复能力；中段提交打通 V1 远端运行和报告；后段提交主要来自真实评测暴露出的问题，每次都把一次失败模式转成代码约束、测试或提示约束。

表 2: 基础节点之后的主要提交

提交	标题	作用
dfb43e9	拆出 SWE 评测状态判定层	新增 record 解析、状态判定和自动评测 driver 的基础能力。
e61088b	添加只读 SWE wave 状态汇总	增加 wave 状态读取脚本，便于并行评测期间只读观察。
0db93c6	补齐 SWE 评测有限损失恢复	新增 worktree checkpoint，周期性保存 patch，支持异常后的有限损失恢复。
d75c77d	修正 SWE Harness 评审发现问题	修复独立评审指出的状态层和恢复层问题，包括兼容性、配对和测试覆盖。
068450e	添加 V1 Pro-Lite 隔离评测入口	新增 <code>swe_v1_prolite_runner.py</code> ，形成 V1 单命令切片入口。

提交	标题	作用
39a66e9 至 c7f727b	V1 远端运行修复组	修复隔离运行时依赖、容器工作目录、远端中断清理、输出权限、测试改动过滤和容器重跑命名。
029b52b	补全 V1 报告缓存摘要	让报告能复用已完成结果并保留更完整的缓存摘要。
68688b0	阻断 V1 重复工具空转	对会话循环加入无进展控制,减少模型在相同工具动作上反复消耗。
3c4a469	收紧 V1 只读阶段工具权限	区分只读角色、coder 角色和测试角色的工具集合,减少角色越界。
4ad403c 至 8c23b03	结构化角色边界修复组	限制硬写入阶段继续读工具、结构化阶段探索预算、空读、提前提交门、违规 patch 路径清理和差异风险阶段空转。
7127933	限制工具调用无进展等待	给工具执行加入无进展等待边界,防止工具层长时间挂住。
a9c8a65 与 d9bdfdc	Go 与 Docker 测试修复	让 <code>run_tests</code> 自动识别 Go 测试目标,并修复 Docker 写入与 Go 测试命令。
167f9a1	修复生成阶段只读空转	修复生成阶段中只读角色继续空转的问题。
c4a7f23 至 dbca8bd	远端代理与结构化超时修复组	修复代理端口占用、备用端口启动速度、健康检查等待、结构化角色超时失效和远端健康检查误判。
6227f0c	修复评测超时取消卡死	新增 <code>abandon_on_timeout</code> ,让调用方在超时后立即返回,防止 provider 取消清理拖住整个流程。
54f0a7d	收紧评测只读角色边界	将结构化角色超时降到 300 秒,并继续收紧只读角色缺工具时的行为。

6 评测过程和结果

V1 的验证不是一次完成的。早期先用 `dry-run` 检查切片、远端目录、报告输出和可评测状态；随后用真实切片跑生成与官方评测；每次暴露框架问题，就回到代码中修复。这个循环让报告中的 `resolved` 数字和工程修复都来自实际运行。

表 3: 已保存的 V1 评测报告

切片	报告文件	生成完成	官方评测完成	resolved	技术失败
26-27	swe_v1_16m_prolite26_27_report.json	2	2	1	0
28-29	swe_v1_16m_prolite28_29_tooltimeout_report.json	2	2	0	0
31	swe_v1_16m_prolite31_goruntests_report.json	1	1	1	0
34-35	swe_v1_16m_prolite34_35_readonlytight_report.json	2	2	1	0

26-27 是 V1 路径的首次真实连通性证明。报告显示两题都完成生成和官方评测，其中第 27 题 resolved，第 26 题 unresolved。这个结果证明脚本可以从切片选择、远端运行、patch 生成、候选身份记录到 official eval 报告写入完成一轮闭环。

28-29 是后续重点调试切片。对应报告 `swe_v1_16m_prolite28_29_tooltimeout_report.json` 显示两题都生成完成并进入官方评测，两个结果都是 unresolved，技术失败为 0。虽然模型没解出题，但这轮很有价值：它暴露了重复工具调用、只读阶段权限、结构化阶段空读、测试 patch 过滤、差异风险阶段过度读取和工具无进展等待等框架问题。随后从 68688b0 到 8c23b03 的一组提交都可以视为对 28-29 评测暴露问题的工程回应。

31 切片验证了 Go 测试识别和 Docker 写入修复。对应报告显示第 31 题生成完成、官方评测完成且 resolved。这个结果配合 a9c8a65 和 d9bdfdc，证明 `run_tests` 对 Go 目标的识别和容器工作目录写入已经走通。

34-35 是最终验证切片。报告 `swe_v1_16m_prolite34_35_readonlytight_report.json` 显示状态为 done，计数为 `generation_done=2`、`eval_done=2`、`resolved=1`、`unresolved=1`、`technical_failed=0`。第 34 题 patch sha 为 73fdd23e339d...，官方评测完成但 unresolved；第 35 题 patch sha 为 a4a90d472657...，官方评测完成且 resolved。第 35 题运行中出现过 `pre-validation-factory` 300 秒超时和 Jest 项目被 `run_tests` 自动识别为 pytest 的 mismatch，但流程没有阻塞，最终预测和官方评测均完成。

7 由评测触发的关键修复

真实评测暴露出的第一类问题是重复空转。模型在只读阶段会继续读取相同文件，或者在已经知道缺少写工具时仍然尝试寻找替代写入方式。这会带来大量 token 消耗，即便脚本没有抛异常，也属于框架 bug。对应修复包括会话层的无进展控制、结构化阶段探索预算、空读限制、提前 `structured_output` 提交门，以及针对 diff-risk 的工具禁用。

第二类问题是工具边界不清。早期角色之间工具能力划分过宽，导致只读角色试图创建临时

probe, 硬写入阶段声明要写文件却继续读取, 验证角色用错测试工具。后续修复把工具集合拆开: 只读角色只有 `file_read` 和 `grep`; coder 拥有 `bash`、`file_read`、`file_write`、`apply_patch`、`run_tests` 和 `grep`; tester 拥有读取、`run_tests`、`grep` 和 `git_diff`, 且 `runner override` 被关闭。

第三类问题是远端运行稳定性。V1 runner 在实际跑题时暴露了远端代理端口占用、备用端口启动慢、健康检查等待过长和健康检查误判。对应提交 `c4a7f23`、`5258595`、`a4d06c2` 和 `dbca8bd` 让代理检查更快、更准确, 也避免因为端口冲突直接失败。

第四类问题是超时无法真正释放调用方。单纯使用 `asyncio.wait_for` 时, 如果底层 provider `coroutine` 在取消清理中长时间不返回, 外层会继续等。提交 `6227f0c` 新增 `abandon_on_timeout`, 用 `asyncio.wait` 到点判断, 如果任务未完成就取消任务、挂上后台结果消费回调, 并立即向调用方抛出 `TimeoutError`。对应测试覆盖了“取消清理永不返回时, 调用方仍按时返回”的场景。

第五类问题是结构化角色缺工具时的行为。提交 `54f0a7d` 将非 coder 结构化角色超时从 900 秒降为 300 秒, 并在 `prompt` 中明确要求缺少写入或命令工具时直接在结构化结果中报告限制。34-35 最终运行中, 第 35 题的 `pre-validation-factory` 在 300 秒处被记录为 `timed out`, 后续角色继续推进, 最终生成和官方评测完成, 这证明该边界在真实 `run` 中生效。

8 复审与验证

这次工作使用了两层验证。第一层是本地或脚本化验证, 包括 `python3 -m compileall`、针对超时和 `workflow` 角色的手工/单测式检查、`git diff --check`, 以及真实 V1 切片评测。部分环境没有安装 `pytest`, 因此某些 Python 单元测试只能通过 `compileall` 和直接导入检查覆盖; 远端 `official eval` 则对生成 `patch` 给出最终 `resolved/unresolved`。

第二层是独立 Agent 复审。早期初版状态层和 V1 入口被复审指出问题后继续修复; `6227f0c` 由 Confucius 复审, 结论为 `PASS`; `54f0a7d` 由 Lovelace 复审, 结论为 `PASS`; 最终 34-35 跑完后, Socrates 对当前 `HEAD`、两次关键提交和最终报告做只读复审, 结论为 `PASS`。Socrates 的意见认为当前仓库现状与提交描述一致, 未发现必须立刻修复的问题; 34-35 报告计数与要求一致; 剩余风险主要是本机报告只含外层摘要, 若要复盘 `runner mismatch` 的每一个内部步骤, 需要继续读取远端 `/nfsEDS/...` 下的 `generation log` 和 `official eval command log`。

9 最终状态

当前分支为 `codex/harness-clean-foundation`, `HEAD` 为 `54f0a7d` 收紧评测只读角色边界。工作树在最终检查时干净。V1 harness 已经具备从切片选择到生成、候选身份记录、官方评测、报告输出的一条可运行路径。它能够在结构化角色超时、测试 `runner mismatch`、远端代理状态变化等边界中继续推进或明确记录状态, 避免把框架问题伪装成模型能力问题。

从评测结果看, V1 当前已经能解决部分任务, 也能稳定记录 `unresolved`。26-27 中 1 题 `resolved`, 28-29 中 0 题 `resolved`, 31 中 1 题 `resolved`, 34-35 中 1 题 `resolved`。更重要的是, 最后一次 34-35 运行满足“发出一条命令并等待完成”的要求: 生成完成、官方评测完成、技术失败为

0，且最终由独立 Agent 给出 PASS。

10 剩余风险与后续建议

剩余风险主要有三点。第一，报告层目前保存的是汇总结果，若要逐步复盘每个角色的内部行为，还需要把远端 trajectory、generation outer log、official eval command log 的关键片段同步到本地归档。第二，run_tests 对前端 Jest 项目的自动识别仍有改进空间；第 35 题虽然最终 resolved，但中途出现过 pytest/Jest mismatch，说明测试工具识别可以继续扩展。第三，结构化角色已经有 300 秒边界，但它们在时间窗口内仍可能消耗较多 token；后续可以进一步用“同一证据重复读取计数”和“同一错误重试计数”压缩单题成本。

这些风险不影响此次 V1 路径的通过结论。它们更适合作为下一轮优化方向：同步更细日志、增强 JS/TS 测试识别、继续降低只读角色成本。

A 报告文件索引

文件	内容
docs/monitoring/swe_v1_16m_prolite26_27_report.json	26-27 真跑 JSON 报告，2 题生成完成、2 题官方评测完成、1 题 resolved。
docs/monitoring/swe_v1_16m_prolite28_29_tooltimeout_report.json	28-29 真跑 JSON 报告，2 题生成完成、2 题官方评测完成、0 题 resolved、0 个技术失败。
docs/monitoring/swe_v1_16m_prolite31_goruntests_report.json	第 31 题真跑 JSON 报告，1 题生成完成、1 题官方评测完成、resolved。
docs/monitoring/swe_v1_16m_prolite34_35_readonlytight_report.json	34-35 最终真跑 JSON 报告，2 题生成完成、2 题官方评测完成、1 题 resolved、0 个技术失败。
docs/monitoring/swe_v1_16m_prolite34_35_readonlytight_report.md	34-35 最终 Markdown 摘要报告。

B 关键命令与验证记录

34-35 最终运行命令已经写入正文。该命令对应远端目录：

```
/nfsEDS/dongyh/data/kaka/docker/opencollab/eval_work/
validation_council_v1_16m_prolite26_35_
formal_34_35_readonlytight_20260708
```

最终报告生成时间为 2026-07-08 09:57:57 +0000。报告中的模型名为 opencollab-glm52-v1-16m-prolite26-35-20260707，workflow 为 validation-council-solve。